A PBL-III (AIM3170) PROJECT REPORT on

# PACE

## Project Allocation, Coordination and Evaluation Portal

Submitted to Manipal University Jaipur

Towards the partial fulfillment for the Award of the Degree of

**B. Tech Computer Science and Engineering**
**(Artificial Intelligence and Machine Learning)**

2026-2027


By

Mihir Sanghvi 2427010544

Krishna Agarwal 2427010030

Mahika Tuteja 2427010439





MANIPAL UNIVERSITY
JAIPUR

Under the guidance of

**Dr. Preeti Narooka**

Team Supervisor


**Department of Artificial Intelligence and Machine Learning**

**Manipal University Jaipur**

**Jaipur, Rajasthan**

MANIPAL UNIVERSITY
JAIPUR

Date: _____

# CERTIFICATE

This is to certify that the project entitled "**PACE (Project Allocation, Coordination and Evaluation)**" is a Bonafide work carried out as part of the course AIM3170, under my supervision from July 2026 to November 2026 by **Mihir Sanghvi (2427010544), Krishna Agarwal (2427010030) and Mahika Tuteja (2427010439)** students of B. Tech Computer Science and Engineering (AIML), 5th Semester at the Department of Artificial Intelligence and Machine Learning, Manipal University Jaipur, during the academic semester 5th in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AIML), at MUJ, Jaipur.

Supervisor: **Dr. Preeti Narooka**

Signature: _____

Department of AIML, Manipal University Jaipur

Signature: _____

HOD, Department of AIML

Manipal University Jaipur

# ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to **Prof. (Dr.) Kuldip Singh Sangwan**, Dean, Faculty of Science, Technology and Architecture, for his constant encouragement, valuable guidance and support throughout the PBL-III course.

We are also deeply thankful to **Dr. Roheet Bhatnagar**, Professor and Associate Dean, Department of Computer Science and Engineering, for his insightful suggestions, academic guidance and continuous motivation, which greatly contributed to the progress of this project.

Our heartfelt thanks go to **Dr. Deepak Panwar**, Associate Professor and Head of the Department, Department of Artificial Intelligence and Machine Learning, for providing the necessary facilities, support and a conducive learning environment that enabled us to carry out this project effectively.

We would like to extend our **special thanks to the PBL Coordinators** for their continuous guidance, coordination and timely support throughout the duration of the project.

We are especially grateful to our project guide, Dr. Preeti Narooka, for the mentorship, feedback and support we received at every stage of the project.

We also thank all faculty members and staff involved in PBL-III for their cooperation and support.

Mihir Sanghvi
Krishna Agarwal
Mahika Tuteja

# ABSTRACT

Project-Based Learning (PBL) asks every student team to find a supervisor, register a project, report its progress every week and be graded on it. When this is coordinated through paper forms, email threads and spreadsheets, signed forms go missing, students do not know where their request stands, supervisors lose track of which weekly reports are still ungraded, and the PBL office has no single view of which teams are still without a supervisor. The objective of this project is to build **PACE (Project Allocation, Coordination and Evaluation)**, a web portal that carries the whole PBL cycle for students, team supervisors and the administrator in one place.

PACE was developed iteratively in weekly increments. The front end is a Next.js 16 and React 19 application written in TypeScript and styled with Tailwind CSS. The back end is Supabase: PostgreSQL for data, Supabase Auth for university email sign-in, and Supabase Storage for signed forms, reports and proof files. Business rules (team size, one group per student, supervisor limits, who may grade) live in the database as security definer functions, and Row Level Security decides what each signed-in user can read. The application is deployed on Vercel.

The portal covers sign-up with verified MUJ email addresses, team formation through team requests, mentor registration with a signed PBL form, eight weekly reports with midterm and endterm submissions, grading out of ten and a 100-point final assessment, queries with attachments, a change of supervisor that both supervisors must approve, formatted announcements, the PBL handout and a directory of 821 MUJ faculty members. Supervisors can post approved student achievements to LinkedIn from one central portal account.

After eight weeks the planned work for Phase 1 is complete and the portal is live. University single sign-on, password recovery, posting as an official LinkedIn Page and analytics that flag teams at risk are left for the next phase.

**Keywords:** project-based learning, web portal, Next.js, Supabase, row level security, workflow automation

# LIST OF TABLES

Table 2.1: Functional requirements ..... 4
Table 2.2: Non-functional requirements ..... 5
Table 2.3: Use case: register the team with a supervisor ..... 6
Table 2.4: Use case: submit and grade a weekly report ..... 6
Table 2.5: Use case: approve and post an achievement ..... 7
Table 2.6: Weekly development plan ..... 7
Table 2.7: Sample test cases ..... 9
Table 3.1: Technology stack ..... 11
Table 3.2: Main database tables ..... 12
Table 3.3: Main database functions ..... 13
Table 3.4: Storage buckets ..... 14
Table A.1: Individual contributions of the team members ..... 23

# LIST OF FIGURES

Figure 2.1: Use case diagram of PACE ..... 6
Figure 2.2: Work completed at the end of each week ..... 8
Figure 3.1: System architecture of PACE ..... 11
Figure 3.2: Main tables of the PACE database ..... 12
Figure 3.3: Team formation through team requests ..... 15
Figure 3.4: States of a mentor request ..... 15
Figure 3.5: Sign-in page with one tab per role ..... 16
Figure 3.6: Student dashboard ..... 17
Figure 3.7: My Group and Mentor page ..... 17
Figure 3.8: Weekly reports page with graded weekly cards ..... 18
Figure 3.9: Student achievements page ..... 18
Figure 3.10: Queries page with the change of supervisor option ..... 19
Figure 3.11: Team supervisor dashboard (My Groups) ..... 19
Figure 3.12: Administrator overview of all groups ..... 20
Figure 3.13: Excellence Award shortlist and the portal LinkedIn account ..... 20

# CONTENTS

ACKNOWLEDGEMENTS ..... i
ABSTRACT ..... ii
LIST OF TABLES ..... iii
LIST OF FIGURES ..... iv
CHAPTER 1: INTRODUCTION ..... 1
1.1 Background and Motivation ..... 1
1.2 Objectives of the Project ..... 1
1.3 Scope of the Work ..... 2
1.4 Product Scenarios ..... 2
1.5 Organization of the Report ..... 3
CHAPTER 2: REQUIREMENT ANALYSIS ..... 4
2.1 Functional Requirements ..... 4
2.2 Non-functional Requirements ..... 5
2.3 Use Case Scenarios ..... 5
2.4 Other Software Engineering Methodologies ..... 7
CHAPTER 3: SYSTEM DESIGN ..... 10
3.1 Design Goals ..... 10
3.2 System Architecture ..... 10
3.3 Detailed Design ..... 11
CHAPTER 4: CONCLUSION ..... 21
4.1 Conclusion ..... 21
4.2 Limitations ..... 21
4.3 Future Work ..... 21
REFERENCES ..... 22
ANNEXURES ..... 23
Annexure A: Individual Contributions ..... 23
Annexure B: Deployment and Test Accounts ..... 23

# CHAPTER 1: INTRODUCTION

## 1.1 Background and Motivation

Project-Based Learning places students in small teams that work on an open problem over a full semester, under the guidance of a faculty supervisor. Research on PBL consistently reports better engagement and deeper understanding when students take ownership of a real project [1], [2], but it also notes that PBL demands more coordination from teachers than lecture-based courses [3]. Every team has to be formed, matched with a supervisor, registered with a signed form, followed week by week and finally graded.

In practice, much of this coordination still happens on paper, over email and in shared spreadsheets. A team lead walks a signed form from one office to another; a supervisor receives weekly reports as attachments scattered across an inbox; the PBL office collects lists of teams by hand to find out who still has no supervisor. Each step is small, but together they cost time and leave students unsure of where they stand. This was the motivation for PACE: one portal where each role sees exactly what it has to do next.

## 1.2 Objectives of the Project

- Give students a single place to form a team, register a supervisor with the signed PBL form, submit weekly reports, see grades and feedback, raise queries and file achievements.
- Give team supervisors one screen for their groups: review mentor requests, read and grade weekly reports, fill in each group's 100-point assessment, grade each student with improvement notes, answer queries and post announcements.
- Give the administrator (PBL office) a view of every group and every student without a group, the power to allot supervisors and to decide change requests that would take a supervisor past the limit, and a shortlist for the Student Excellence Award.
- Enforce the PBL rules in the database itself, so that no user interface mistake can break them.
- Keep the portal free to run and easy to maintain by a small team, using managed services.

## 1.3 Scope of the Work

Phase 1 covers the complete PBL cycle for one semester in the Department of Artificial Intelligence and Machine Learning. The portal has three role-based portals (Student, Team Supervisor and Administrator) on one code base. In scope are: account creation with university email addresses and email verification; team formation; mentor registration with the signed form; eight weekly reports with midterm and endterm submissions; report grading and the 100-point assessment; individual grades; queries with attachments; change of supervisor; announcements with attachments; notifications; the PBL handout; a faculty directory; and student achievements with LinkedIn posting.

Out of scope for Phase 1 are: single sign-on with the university identity provider, password recovery by email, analytics that predict which teams are at risk, and posting as an official LinkedIn Page, which needs separate approval from LinkedIn.

## 1.4 Product Scenarios

The following scenarios describe how the three roles use PACE in a typical week.

**Scenario 1: Forming a team.** Mihir signs up with a university email address and confirms the link that arrives in the inbox, then creates a group, enters the project idea and sends a team request to each of two classmates by registration number. Krishna sees the request under Requests and accepts it; any other pending requests for Krishna are declined automatically.

**Scenario 2: Registering a supervisor.** After talking to the supervisor in person, the team lead uploads the signed PBL form and sends it to that supervisor. The supervisor opens the form inside the portal, notices a missing signature and clicks Ask for changes with a short note. The lead uploads a corrected form; the supervisor approves it and the group is now registered with the supervisor. All three students are notified.

**Scenario 3: Weekly reporting.** Each week one member submits the team's report with a summary and any files. The supervisor grades it out of ten with feedback, and the team sees the grade on the Weekly Reports page. After week 3 the team uploads its midterm PPT and synopsis, and after week 7 the endterm set. At the end of the semester the supervisor opens Final Grading, fills in the 100-point plan and publishes it once all of their groups are graded.

**Scenario 4: Celebrating an achievement.** A student wins a hackathon and files it as an achievement with the certificate as proof. The supervisor approves it with bonus marks,

checks the LinkedIn preview and posts it from the portal's central account in one click. The administrator sees the student move up the Excellence Award shortlist.

**Scenario 5: Changing supervisor.** Team 1 wants to move from Dr. A to Dr. B. The team lead raises a change request, picks Dr. B and attaches the supporting files. Both supervisors are asked. If Dr. B has fewer than seven groups and both approve, the group moves at once and the PBL office is told. If Dr. B already has seven, the request goes to the administrator after both approve, who can let Dr. B take one more group or decline. If either supervisor declines, the request closes and the team and the other supervisor see the reason.

## 1.5 Organization of the Report

Chapter 2 analyses the requirements: functional and non-functional requirements, use case scenarios and the development methodology. Chapter 3 presents the system design: design goals, architecture, database and security design, the main workflows and the user interface. Chapter 4 concludes the report and lists the work planned for the next phase. References and annexures follow.

# CHAPTER 2: REQUIREMENT ANALYSIS

Requirements were gathered from the PBL guidelines of the department, the paper forms used in earlier semesters, discussions with our supervisor, and our own experience as students in PBL-I and PBL-II. They were written as short, testable statements following the practice described by Sommerville [4].

## 2.1 Functional Requirements

Table 2.1 lists the functional requirements, grouped by the role that uses them.

Table 2.1: Functional requirements

| ID | Requirement | Role |
|---|---|---|
| FR1 | Sign up with a university email address; the address decides the role; the account is active only after the email is verified. | All |
| FR2 | Create a group with a project idea; the creator becomes team lead. | Student |
| FR3 | Send a team request to each teammate; only signed-up students; a team has at most three members; accepting one request declines the others. | Student |
| FR4 | Upload the signed PBL form and send it to a chosen supervisor; at most three open requests at a time. | Student |
| FR5 | Approve, ask for changes on, or reject a mentor request; a supervisor takes at most seven groups. | Supervisor |
| FR6 | Submit one report per week for eight weeks, with a summary and files; upload the midterm PPT and synopsis after week 3 and the endterm set after week 7. | Student |
| FR7 | Grade weekly reports out of 10 with feedback and each student with improvement notes; fill in the group's 100-point assessment and publish it once every group is graded; only the group's own supervisor may grade. | Supervisor |
| FR8 | Raise a query with attached files and receive replies; ask to move to a named supervisor, which needs both the current and the new supervisor to approve. | Student |
| FR9 | Post announcements with formatting and attachments to own groups or to all students. | Supervisor, Admin |
| FR10 | File an achievement with proof; review it with bonus marks; post approved achievements to LinkedIn. | All |
| FR11 | See every group and the students without one; allot supervisors at random; move a group with a reason; decide change requests where the new supervisor already has seven groups. | Admin |

| ID | Requirement | Role |
|---|---|---|
| FR12 | Notify the right people automatically on every request, decision, report, grade and announcement. | System |
| FR13 | Search the MUJ faculty directory by name, email or expertise, filtered by department. | All |
| FR14 | Open the PBL handout inside every portal. | All |

## 2.2 Non-functional Requirements

Table 2.2: Non-functional requirements

| Category | Requirement |
|---|---|
| Security | Every table protected by Row Level Security; all changes through database functions that check the caller; private files served through short-lived signed links; LinkedIn token encrypted on the server. |
| Integrity | Rules such as team size, one group per student and the supervisor limit are enforced by the database, with row locks so that two supervisors cannot approve the same group at once. |
| Usability | Plain-English labels, one clear next step on every dashboard, in-portal document viewer, drag-and-drop uploads, and pages that work on phones and laptops. |
| Performance | Pages load in under two seconds on a campus connection; data is fetched directly from Supabase without an extra server hop. |
| Availability | Hosted on Vercel and Supabase managed services; no server to maintain. |
| Maintainability | TypeScript throughout; every database call in one repository file; schema changes kept as ordered SQL migrations. |
| Cost | Runs on the free tiers of Vercel and Supabase for a department-sized load. |

## 2.3 Use Case Scenarios

Figure 2.1 shows the three actors and their main use cases. The student starts most flows; the team supervisor reviews and grades; the administrator oversees all groups and handles exceptions.

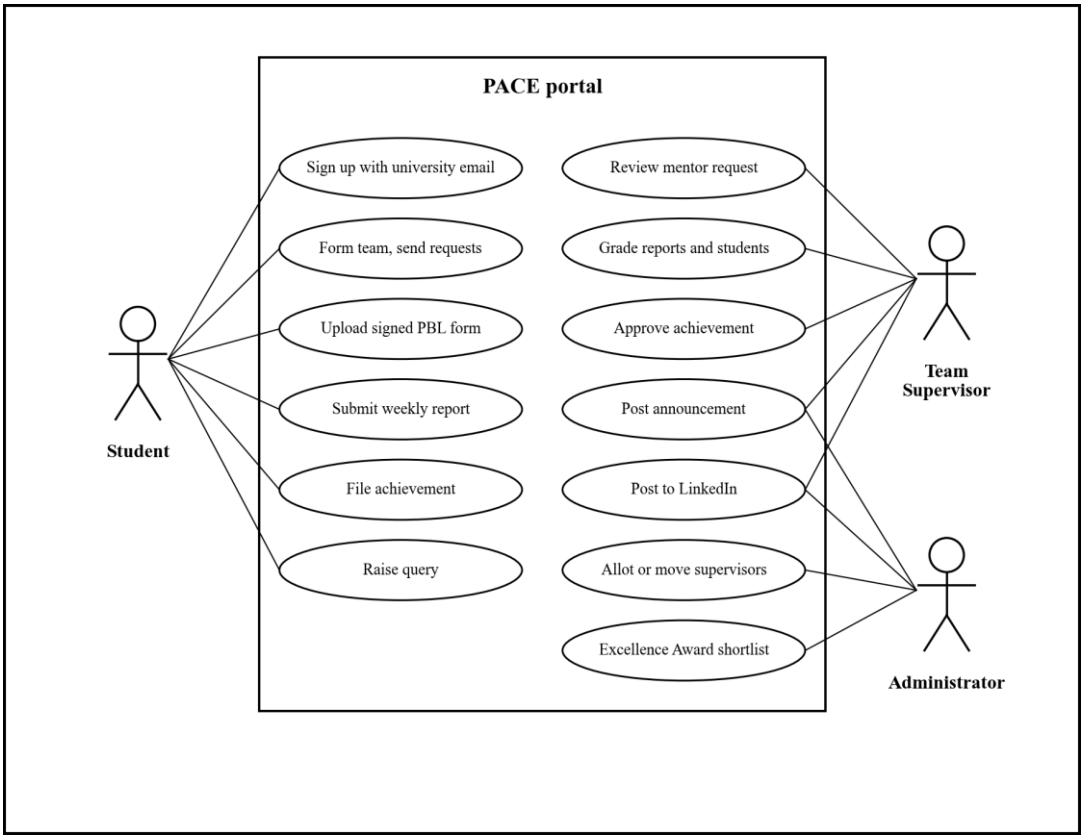



Figure 2.1: Use case diagram of PACE

Tables 2.3 to 2.5 describe the three most important use cases in detail.

Table 2.3: Use case: register the team with a supervisor

| Field | Description |
|---|---|
| Actors | Team lead (primary), team supervisor |
| Precondition | The lead has a group; the supervisor has signed the PBL form offline. |
| Main flow | 1. Lead picks the supervisor by name and uploads the signed form.<br>2. The supervisor is notified and opens the form in the portal.<br>3. The supervisor approves; the group is registered to the supervisor.<br>4. Every member is notified; the group's other open requests are closed. |
| Alternate flows | Ask for changes: a note is required; the lead uploads a corrected form and the request returns to review.<br>Reject: the lead may send the form to another supervisor. |
| Postcondition | The group has exactly one supervisor. |

Table 2.4: Use case: submit and grade a weekly report

| Field | Description |
|---|---|
| Actors | Any group member (primary), team supervisor |
| Precondition | The group has a supervisor; the week's slot is open. |

| Field | Description |
|---|---|
| Main flow | 1. A member writes the summary and attaches files.<br>2. The supervisor is notified and the report count on the dashboard goes up.<br>3. The supervisor grades it out of 10 and adds feedback.<br>4. The group sees the grade and feedback. |
| Alternate flows | The report can be replaced until it is graded. |
| Postcondition | The week's report is stored with its grade. |

Table 2.5: Use case: approve and post an achievement

| Field | Description |
|---|---|
| Actors | Student (primary), team supervisor, administrator |
| Precondition | The student's group has a supervisor; the portal LinkedIn account is connected. |
| Main flow | 1. Student files the achievement with proof files and co-participants.<br>2. Supervisor opens it, sets bonus marks and opens the LinkedIn preview.<br>3. Supervisor edits the text if needed and clicks Approve and post to LinkedIn.<br>4. The achievement is approved and posted; the student is notified with a link. |
| Alternate flows | Ask for more details; reject with a reason; approve without posting; remove the post later and post again. |
| Postcondition | The achievement counts towards the Excellence Award shortlist. |

## 2.4 Other Software Engineering Methodologies

### *2.4.1 Development model*

We followed an iterative and incremental model with one-week iterations, in the spirit of agile development [5], [6]. Each week began with a short plan agreed with the supervisor, produced a working, deployed increment, and ended with the weekly progress report. Table 2.6 shows the plan as it was carried out, and Figure 2.2 the share of work completed at the end of each week.

Table 2.6: Weekly development plan

| Week | Work done | Done |
|---|---|---|
| I | Problem study, requirements, choice of technology stack, interface sketches | 10% |
| II | Database design, landing and sign-in screens, portal layout | 25% |

| Week | Work done | Done |
|---|---|---|
| III | Next.js project and Vercel deployment, sign-in, student dashboard, project and profile pages, sample data | 40% |
| IV | Move to Supabase; groups, weekly reports, grading, queries; faculty directory; sign-up with email verification | 55% |
| V | Mentor registration with signed form; change of supervisor; database notifications | 70% |
| VI | Achievements; LinkedIn posting from a central account; drag-and-drop uploads; PACE branding | 85% |
| VII | Team requests; supervisor-only grading; formatted announcements; new theme; test accounts; first draft of this report | 95% |
| VIII | Eight weekly report slots with midterm and endterm uploads; 100-point assessment and Final Grading; query attachments; change of supervisor approved by both supervisors; notification pages; PBL handout; several files per upload; final report | 100% |

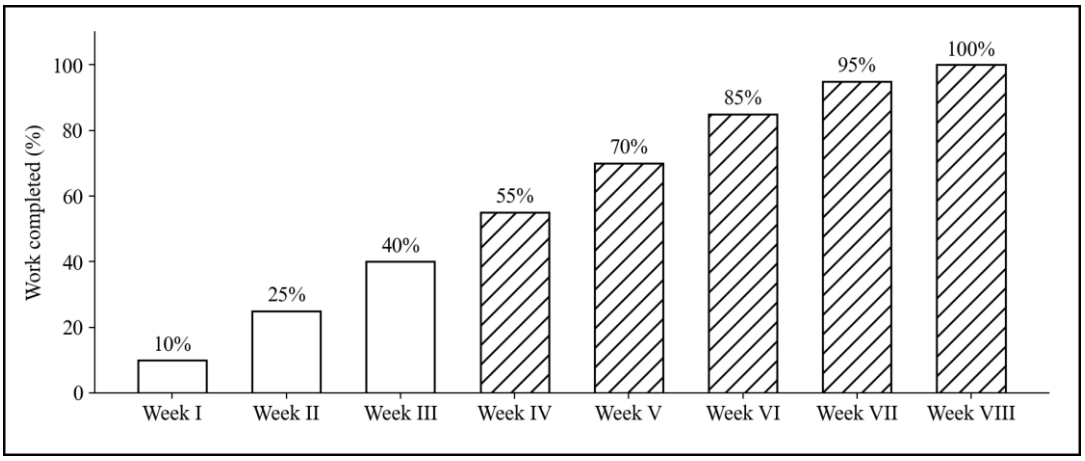



Figure 2.2: Work completed at the end of each week

### *2.4.2 Version control and deployment*

All code is kept in a Git repository. Work is committed on a development branch and then fast-forwarded to the main branch, which Vercel builds and deploys automatically. Database changes are written as numbered SQL migration files so that the schema can be rebuilt from scratch in order.

### *2.4.3 Testing*

Each feature was tested manually against written test cases using five test accounts: three students, a team supervisor and an administrator. Because the rules live in the database, each test also tried to break a rule directly (for example, a fourth member joining a team) and checked that the database refused it. Table 2.7 shows a sample of the test cases.

Table 2.7: Sample test cases

| ID | Test | Expected result | Result |
|---|---|---|---|
| T1 | Sign up with a non-university email | Sign-up refused | Pass |
| T2 | Send a team request to a student who has not signed up | Request refused with a clear message | Pass |
| T3 | Accept a request while holding two others | Joined; the other two declined | Pass |
| T4 | Add a fourth member to a team | Refused by the database | Pass |
| T5 | Supervisor asks for changes without a note | Note required | Pass |
| T6 | Two supervisors approve the same group together | Only the first succeeds | Pass |
| T7 | Administrator tries to grade a weekly report | Refused; grades are read-only | Pass |
| T8 | Paste an announcement from Outlook or Word | Bold, lists and tables kept | Pass |
| T9 | Remove a LinkedIn post and post again | Achievement returns to Action needed, still approved | Pass |
| T10 | Both supervisors approve a move to a supervisor with fewer than seven groups | Group moves at once; administrator notified | Pass |
| T11 | Both supervisors approve a move to a supervisor who already has seven groups | Request waits for the administrator; group moves only after approval | Pass |
| T12 | The requested supervisor declines with a reason | Request closed; team and current supervisor see the reason | Pass |

# CHAPTER 3: SYSTEM DESIGN

## 3.1 Design Goals

- **Rules in one place.** Every PBL rule is written once, in the database, so the three portals cannot disagree and no request can bypass it.
- **Least privilege.** A signed-in user can read only what Row Level Security allows and can change data only through functions that check who is calling.
- **One next step.** Each dashboard shows what the user has to do now: a request to answer, a report to grade, a form to correct.
- **Simple to run.** No custom server for data; managed hosting for the database and the web application.
- **Clear and accessible screens.** Plain-English copy, readable contrast, and layouts that work on phones.

## 3.2 System Architecture

PACE is a client-server web application (Figure 3.1). The client is built with Next.js 16 [7] using the App Router and React 19 [8], styled with Tailwind CSS v4 [9]. Each of the three portals is a folder of pages wrapped by a shared shell with a sidebar and top bar. The pages never talk to the database directly: every call goes through a single repository module that maps database rows to typed objects.

The back end is Supabase [10], which provides PostgreSQL, authentication and file storage behind one API. The client calls Supabase over HTTPS with the user's session token, so every query runs as that user and PostgreSQL Row Level Security [11] decides which rows are visible. The only server code of our own is a small set of route handlers on Vercel that talk to LinkedIn, because they hold the LinkedIn client secret. Documents open inside the portal through a viewer built on PDF.js [12].

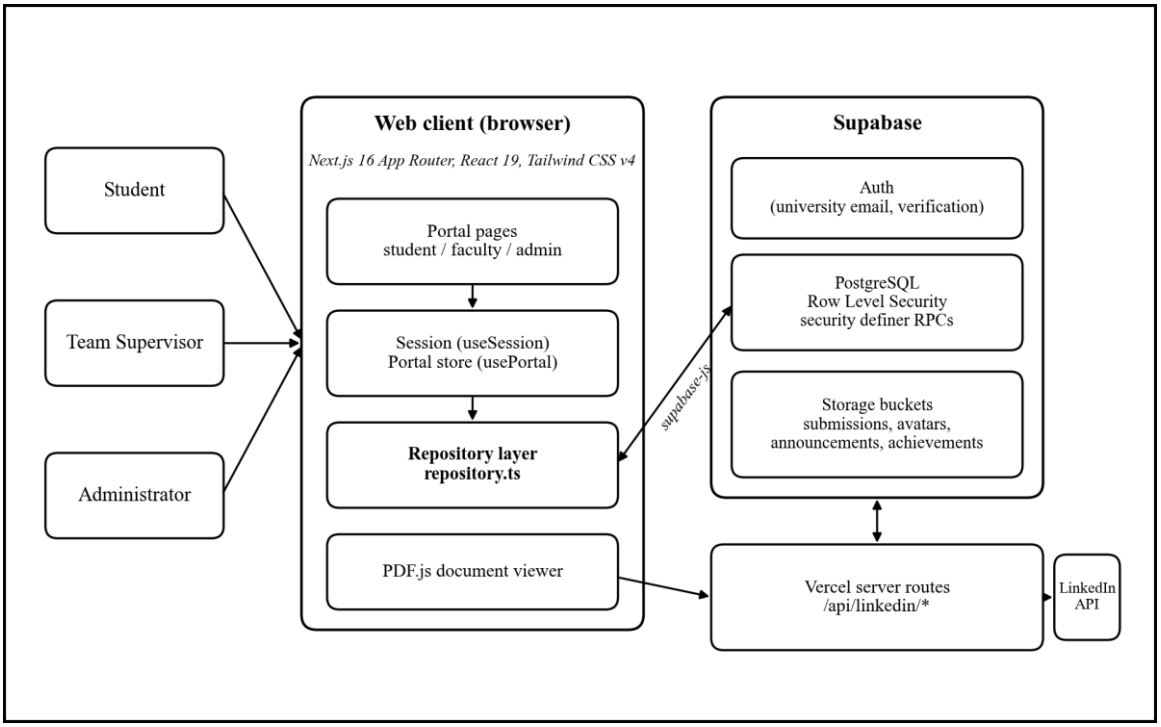



Figure 3.1: System architecture of PACE

Table 3.1: Technology stack

| Layer | Technology | Purpose |
|---|---|---|
| Front end | Next.js 16, React 19, TypeScript | Pages for the three portals, routing, client state |
| Styling | Tailwind CSS v4 | Design tokens, responsive layout |
| Authentication | Supabase Auth | University email sign-up, email verification, sessions |
| Database | PostgreSQL (Supabase) | Data, Row Level Security, rule-checking functions, triggers |
| File storage | Supabase Storage | Signed forms, reports, proof files, avatars |
| Server routes | Next.js route handlers on Vercel | LinkedIn connection and posting |
| Documents | PDF.js | In-portal viewer for PDFs and images |
| Data import | Python | Scraping the MUJ faculty directory |
| Hosting | Vercel | Build and deployment from the main branch |

## 3.3 Detailed Design

### *3.3.1 Database design*

Figure 3.2 shows the main tables and how they relate. A profile exists for every signed-in user and carries the role. Students and faculty have their own tables; the faculty table was

filled from the university website (821 entries) so that a supervisor's account links to their existing row when they sign up. A group belongs to at most one faculty member; group_members links students to groups and marks the lead; group_invitations holds team requests. mentor_requests, weekly_reports, student_grades, achievements and notifications hang off groups and students. group_milestones and group_grades hold the midterm and endterm uploads and the 100-point assessment, and supervisor_changes records every move of a group. Table 3.2 summarises them.

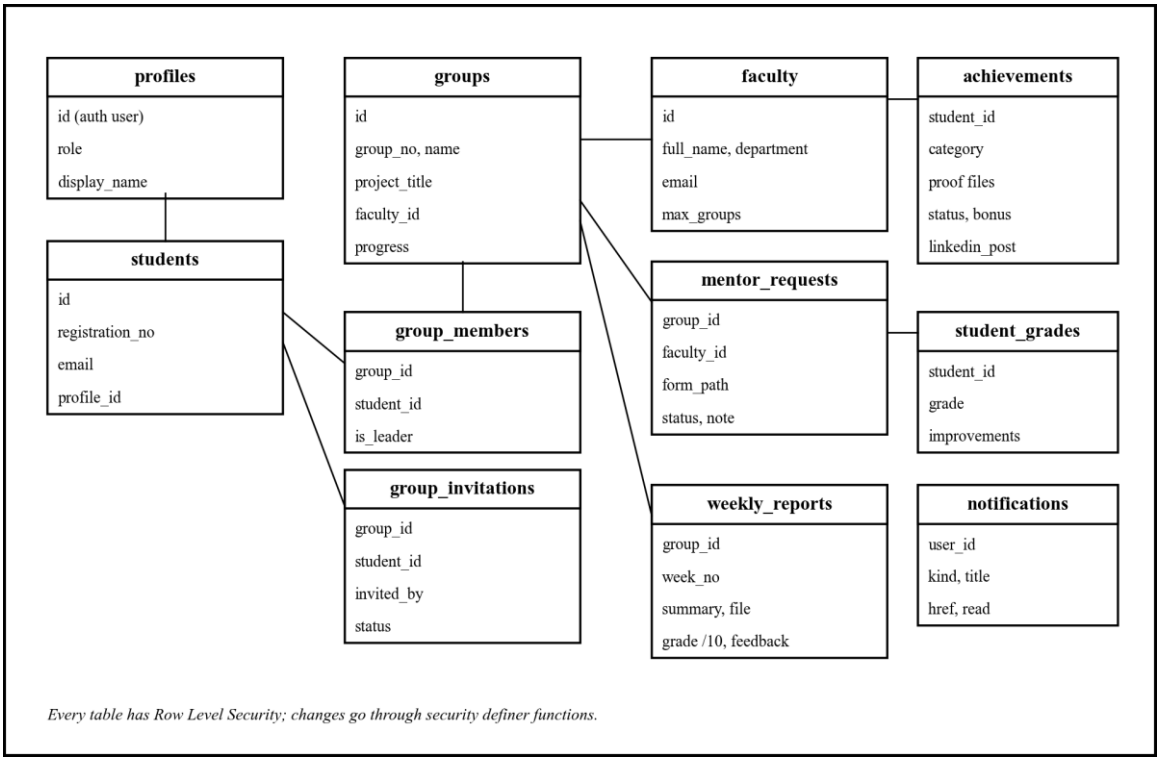



Figure 3.2: Main tables of the PACE database

Table 3.2: Main database tables

| Table | Holds |
|---|---|
| profiles | One row per signed-in user: role and display name |
| students / faculty | Student roster (registration number, email) and MUJ faculty directory (department, expertise, group limit) |
| groups, group_members | Teams, their project, supervisor and progress; membership with the team lead flag |
| group_invitations | Team requests with status pending, accepted, declined or cancelled |
| mentor_requests | Signed PBL form sent to a supervisor, with status and the supervisor's note |
| weekly_reports | One report per group per week (eight by default), with files, grade out of 10 and feedback |

| Table | Holds |
|---|---|
| group_milestones | Midterm (after week 3) and endterm (after week 7) PPT and synopsis uploads |
| group_grades | The 100-point assessment of each group, and whether students can see it yet |
| supervisor_changes | Every move of a group from one supervisor to another, with the reason |
| student_grades | Individual grades and improvement notes, including achievement bonus marks |
| achievements | Achievement details, proof files, co-participants, review status and LinkedIn post link |
| tickets, announcements | Student queries and change-of-supervisor requests with replies and attached files; announcements with formatted text and attachments |
| notifications | Per-user activity feed created by database triggers |

### *3.3.2 Security design*

Security follows the OWASP guidance on broken access control [13]. The browser holds only the public Supabase key; what a user can do is decided entirely by the database. Every table has Row Level Security: students see only their own group, supervisors only the groups they supervise, and the administrator everything. The client is allowed to read under these policies but not to write directly; every change goes through a **security definer** function that first checks who is calling (for example, that the caller is the team lead, or the group's own supervisor). Private helper functions such as my_student_id(), require_leader() and is_staff() live in a separate schema that clients cannot call. Table 3.3 lists the main functions.

Table 3.3: Main database functions

| Function | What it does and checks |
|---|---|
| create_group | Creates a group with the caller as lead; refuses if the caller is already in a group |
| invite_member, respond_to_invitation | Team requests: only signed-up students, at most two teammates, accepting declines other requests |
| request_mentor, resubmit_mentor_request | Team lead sends or corrects the signed form; at most three open requests |
| review_mentor_request | Supervisor approves, asks for changes (note required) or rejects; row locks stop double approval; seven-group limit |

| Function | What it does and checks |
|---|---|
| auto_allocate_mentors, change_supervisor | Administrator allots a random supervisor with fewer than seven groups, or moves a group directly with a reason |
| raise_supervisor_change, approve_supervisor_change, decline_supervisor_change | Team lead names the new supervisor; both supervisors must approve; the group moves at once unless the new supervisor already has seven groups |
| admin_approve_supervisor_change, admin_decline_supervisor_change | Administrator decides a change where the new supervisor is already full |
| grade_report, grade_student | Only the group's own supervisor may grade |
| submit_milestone | Uploads the midterm or endterm PPT and synopsis for the group |
| save_group_grades, publish_group_grades | Supervisor saves the 100-point assessment; publishing is refused until every group of that supervisor is graded |
| submit_achievement, review_achievement | Student files; supervisor approves with bonus marks, asks for details or rejects |
| record_linkedin_post, unrecord_linkedin_post | Records or withdraws the LinkedIn post of an approved achievement |

Files are stored in private buckets (Table 3.4) and opened through signed links that expire. The central LinkedIn access token is encrypted with AES-256-GCM [14] using a key that exists only on the server, so even the database administrator cannot read it.

Table 3.4: Storage buckets

| Bucket | Access | Contents |
|---|---|---|
| submissions | Private, signed links | Weekly reports, midterm and endterm files, signed PBL forms and query attachments, by group |
| achievements | Private, signed links | Proof files for achievements |
| announcements | Private, signed links | Announcement attachments |
| avatars | Public | Profile pictures |

### *3.3.3 Workflow design*

Two workflows carry most of the rules. Team formation (Figure 3.3) is driven by individual requests: the lead sends one request per teammate, and the database guarantees that a student ends up in exactly one group and that a team never exceeds three members. Mentor registration (Figure 3.4) is a small state machine: a request starts Under review and moves to Approved, Changes requested or Rejected. A corrected form returns it to Under review.

When one supervisor approves, the group's other open requests are closed automatically. Every transition creates notifications for the people involved through database triggers, so no one has to remember to send them.

Two newer workflows follow the same pattern. A change of supervisor needs two approvals: one from the current supervisor and one from the supervisor the team asked for. When the second approval arrives, the database counts the new supervisor's groups. Below seven, the group moves straight away and the administrator is only informed. At seven, the request waits for the administrator, who can allow the extra group or decline. Assessment has two steps. Each weekly report is graded out of ten. At the end, the supervisor fills in the 100-point plan: synopsis 10 (the average of the two synopsis marks), weekly progress 15, analysis PPT 15, design and implementation 20, and the end-term exam 40. The weekly figure starts as the average report grade scaled to 15 and can be changed. Students see the total only after the supervisor publishes it, and the database refuses to publish until all of that supervisor's groups are graded.

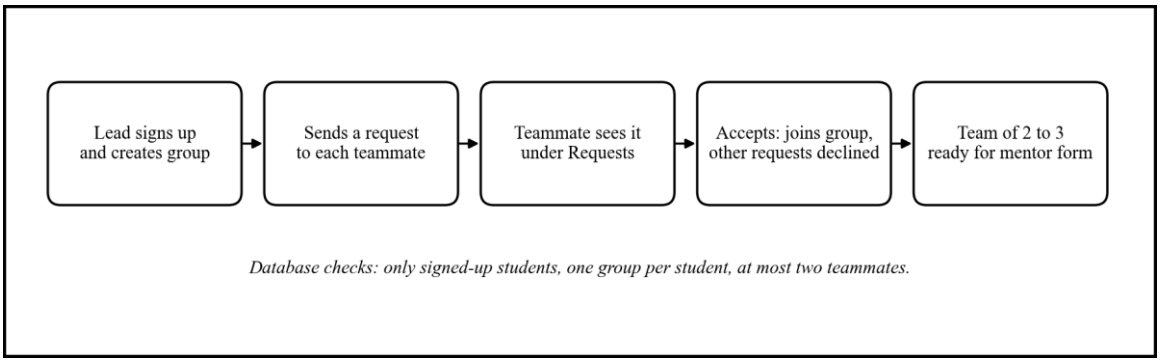



Figure 3.3: Team formation through team requests

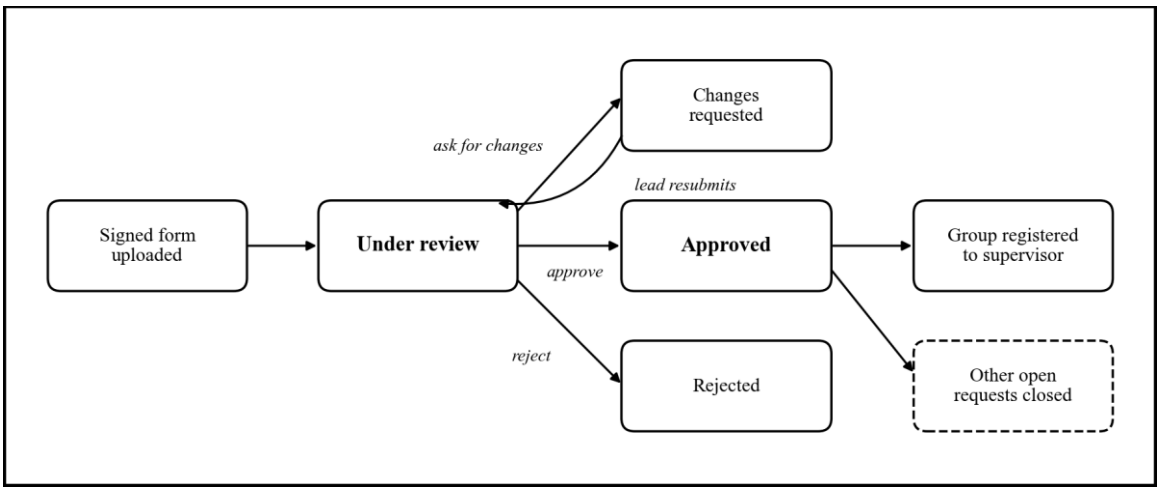



Figure 3.4: States of a mentor request

### *3.3.4 LinkedIn integration*

Approved achievements are posted from one central LinkedIn account that the administrator connects once through LinkedIn's own sign-in page using OAuth 2.0, so its password never passes through the portal [15]. When a supervisor approves an achievement, the portal writes a post from the achievement's template and shows it as LinkedIn will draw it. The supervisor can edit it and post it in one click. The Vercel route handler forwards the user's Supabase token, checks that the user may post, decrypts the LinkedIn token and calls LinkedIn's API. A post can later be removed, which sends the achievement back to Action needed so it can be posted again.

### *3.3.5 User interface design*

Each portal opens on a dashboard that answers one question: what needs my attention now? Pages share one set of components (cards, buttons, badges, empty states, upload boxes) so that the three portals feel the same. The screens use a light blue and white theme, watercolour drawings of the campus and serif headings. Figures 3.5 to 3.13 show the main screens of the deployed portal in colour.

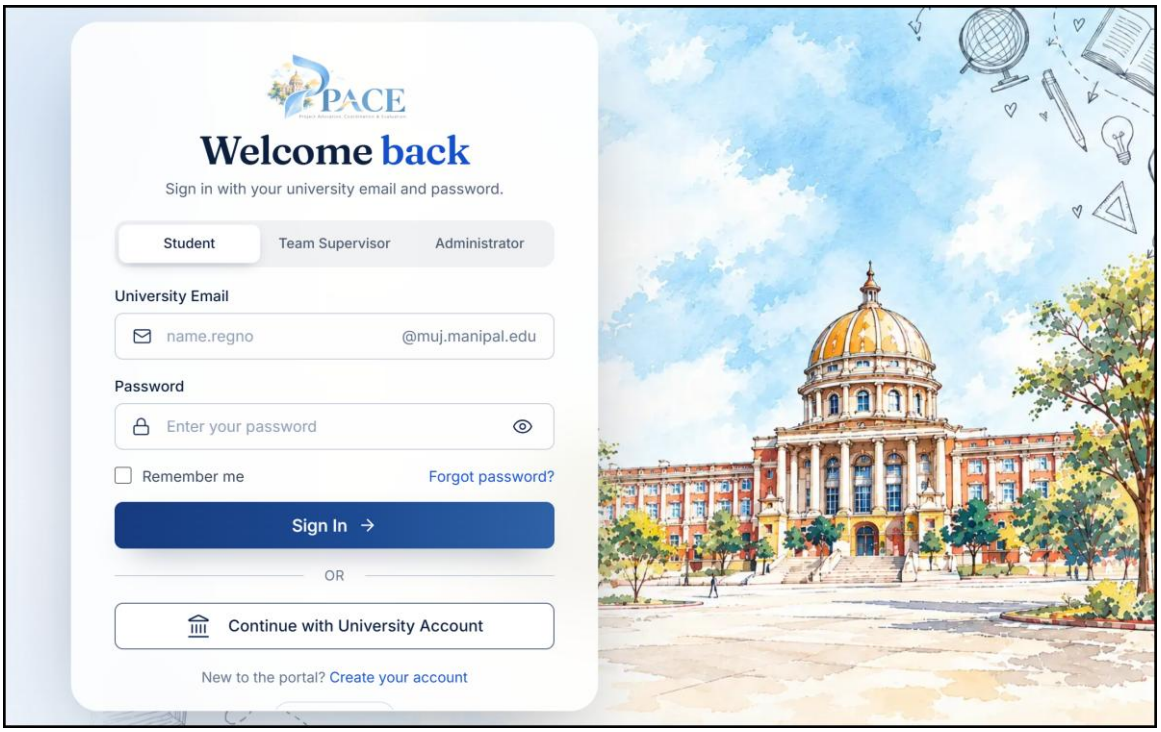



Figure 3.5: Sign-in page with one tab per role

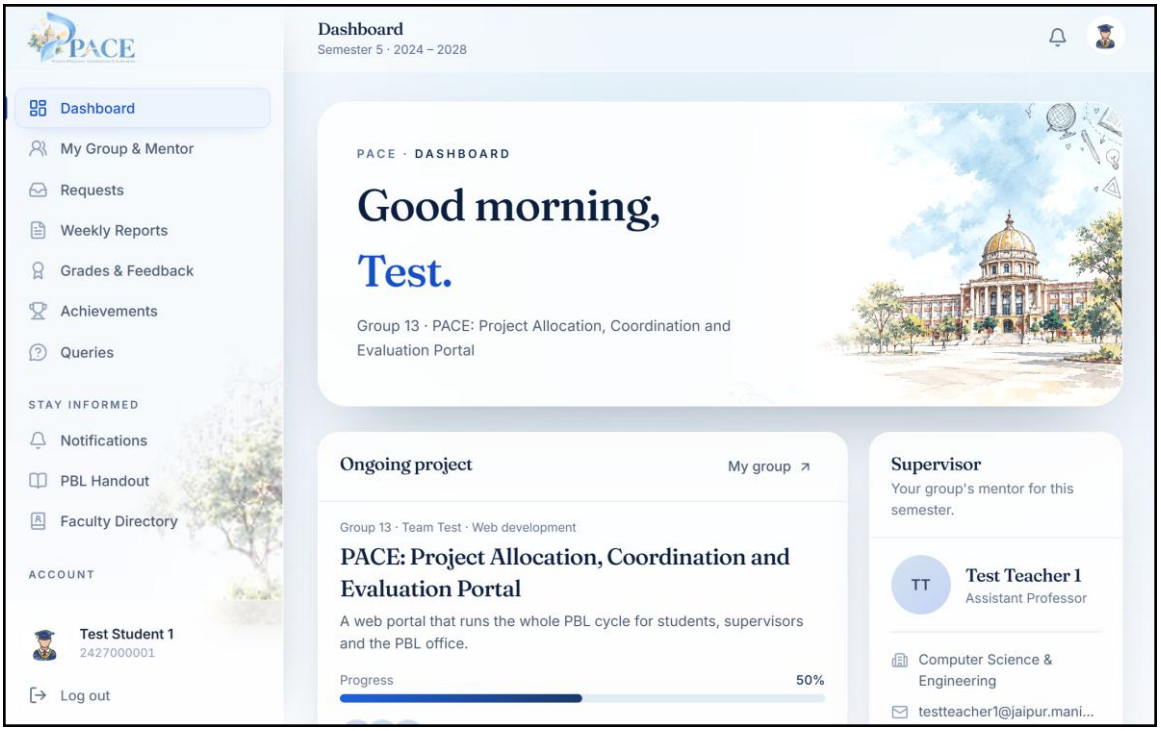



Figure 3.6: Student dashboard

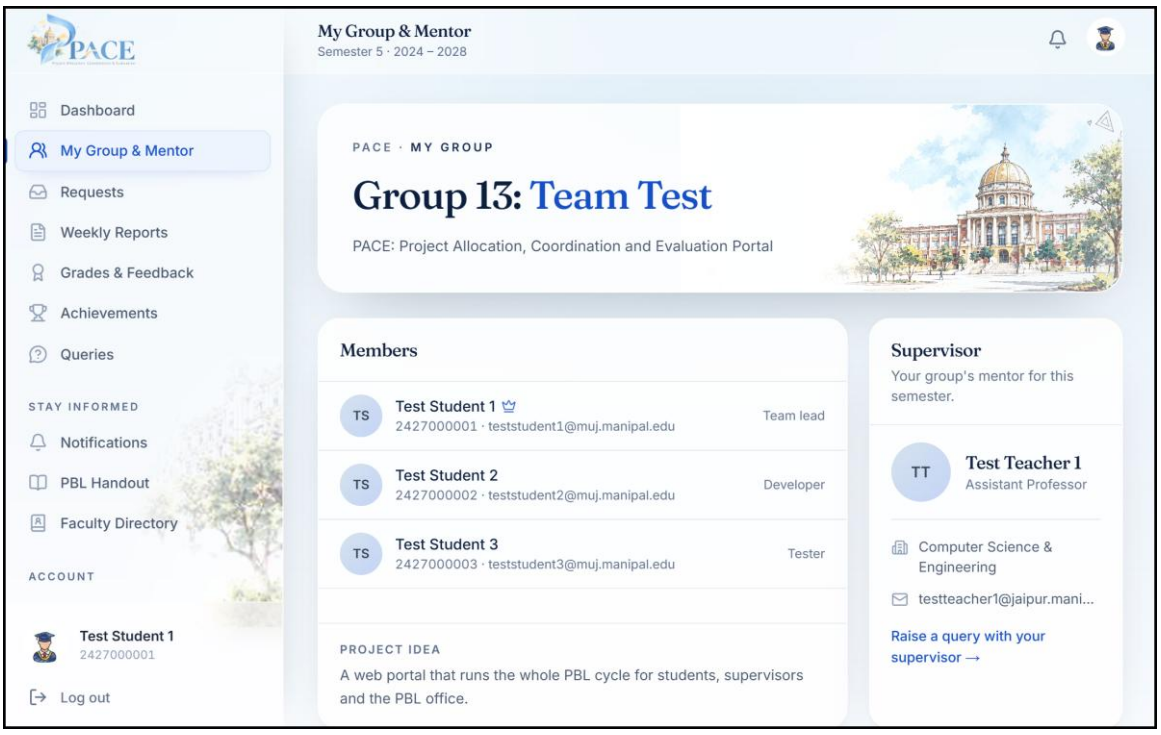



Figure 3.7: My Group and Mentor page

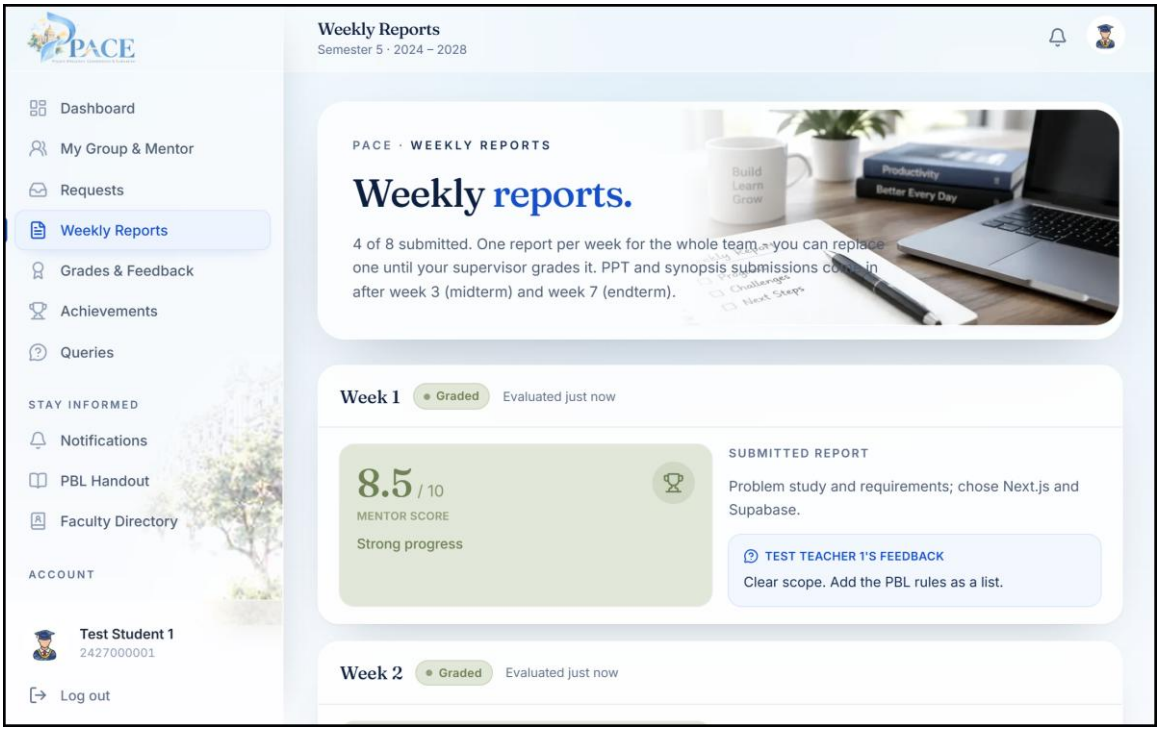



Figure 3.8: Weekly reports page with graded weekly cards

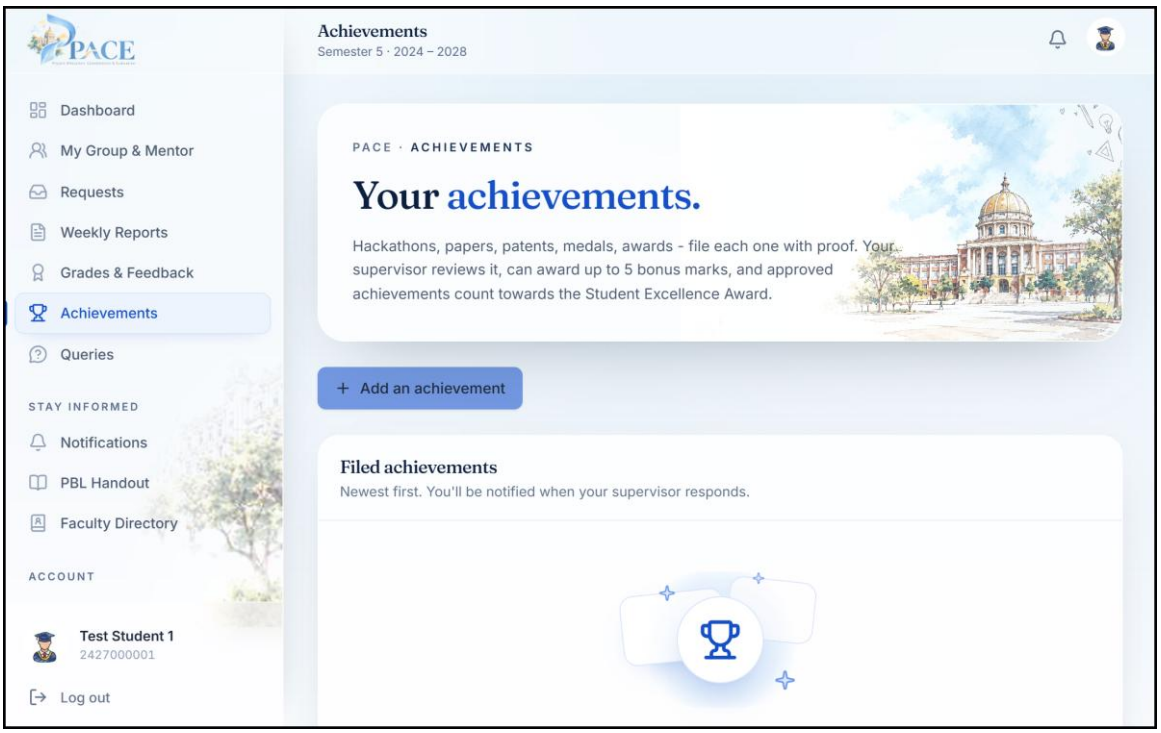



Figure 3.9: Student achievements page

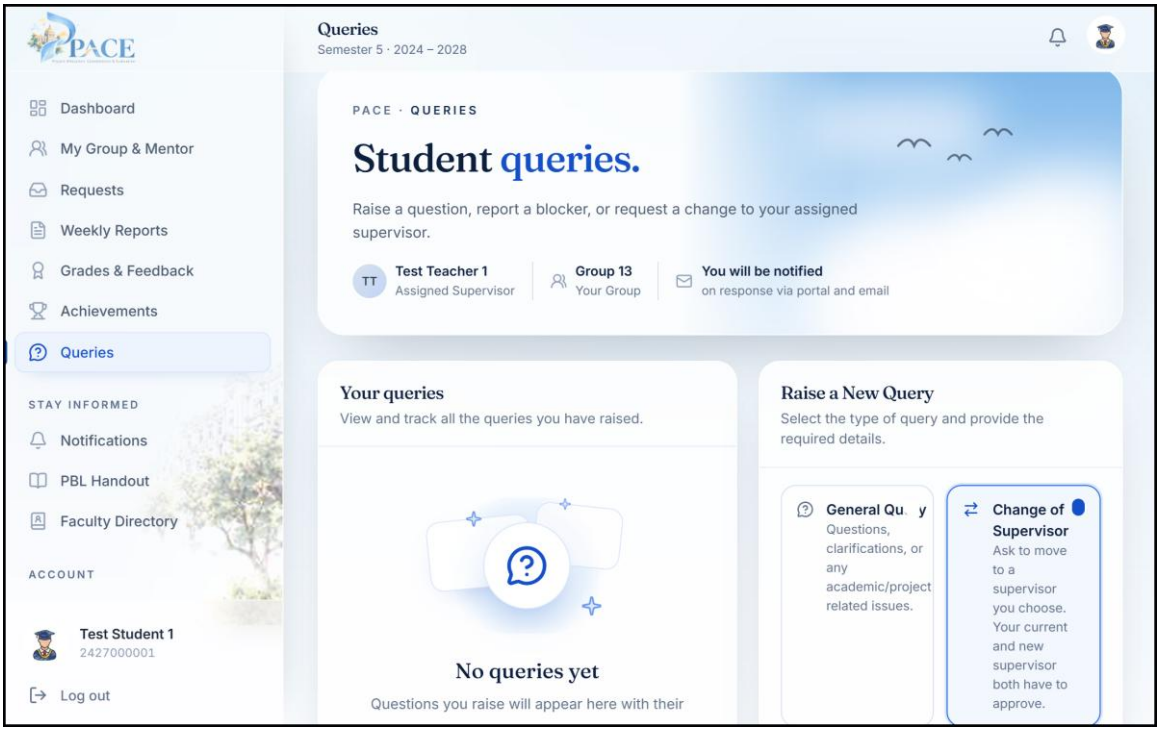



Figure 3.10: Queries page with the change of supervisor option

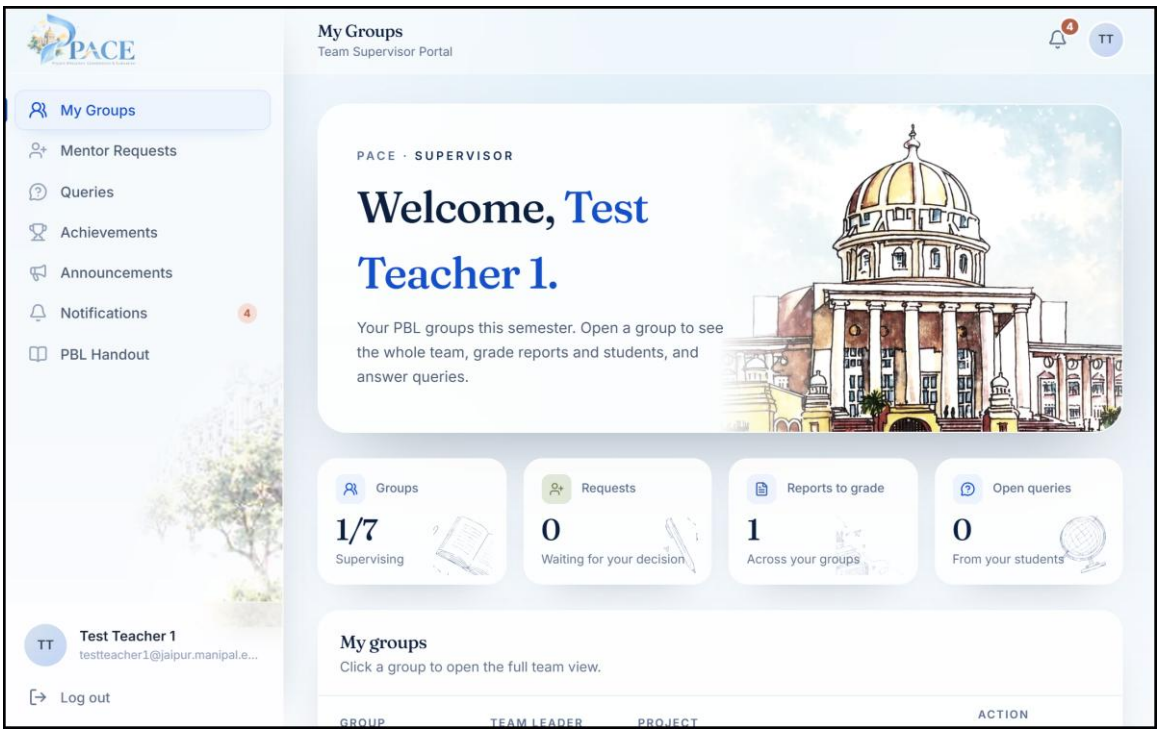



Figure 3.11: Team supervisor dashboard (My Groups)

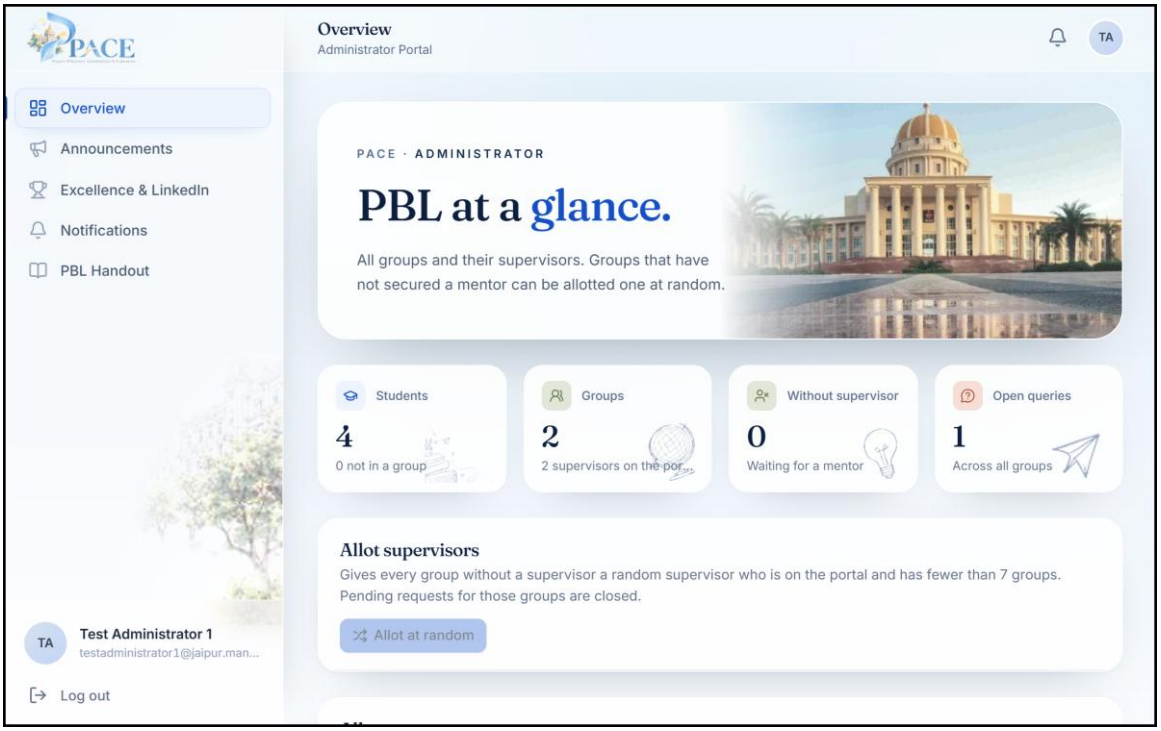



Figure 3.12: Administrator overview of all groups

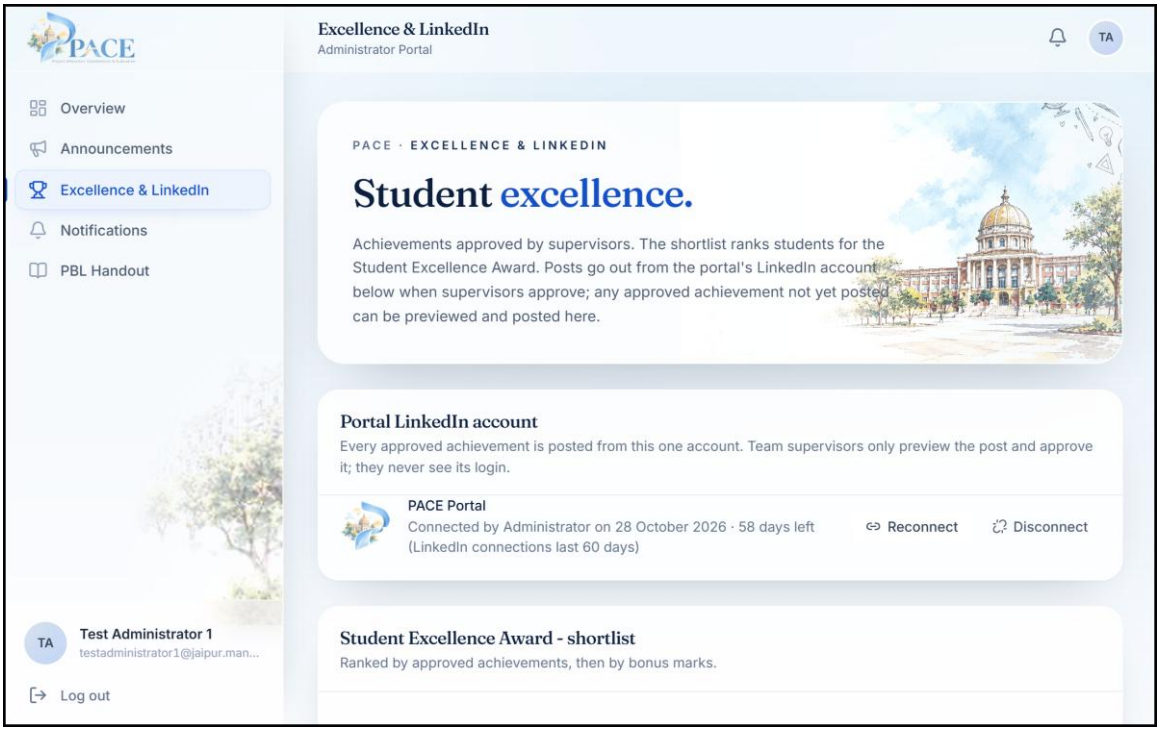



Figure 3.13: Excellence Award shortlist and the portal LinkedIn account

# CHAPTER 4: CONCLUSION

## 4.1 Conclusion

Phase 1 of PACE set out to move the PBL cycle from paper forms, email and spreadsheets into one portal for students, team supervisors and the administrator. After eight weeks the planned work is complete and the portal is live. Students form teams, register a supervisor with the signed PBL form, submit eight weekly reports and the midterm and endterm files, see their grades, raise queries with attachments, ask to change supervisor and file achievements. Supervisors review mentor requests, grade reports, fill in the 100-point assessment, approve or decline change requests, answer queries, post announcements and publish student achievements to LinkedIn. The administrator sees every group and every student without one, allots supervisors, decides change requests that go past a supervisor's limit, and has a ready shortlist for the Student Excellence Award.

The most important design decision was to place every rule in the database, behind Row Level Security and rule-checking functions. This kept the three portals consistent, made the rules testable on their own, and means that a mistake in a page cannot let a student join two teams or let the wrong person grade a report.

## 4.2 Limitations

- Sign-in uses university email and password; single sign-on with the university account is not yet available, nor is password recovery.
- Supervisors must sign up before they can review requests; until then the administrator allots groups.
- Posts go out from a LinkedIn member profile; posting as the official university Page needs LinkedIn's approval.

## 4.3 Future Work

- University single sign-on and password recovery.
- Analytics: a health score for each project, early warning for teams that miss reports, and progress prediction.
- A pilot with one section of students, and export of the 100-point marks in the university's mark sheet format.

# REFERENCES


[1] J. W. Thomas, “A review of research on project-based learning,” The Autodesk Foundation, San Rafael, CA, USA, 2000.

[2] D. Kokotsaki, V. Menzies and A. Wiggins, “Project-based learning: A review of the literature,” Improving Schools, vol. 19, no. 3, pp. 267-282, 2016.

[3] J. S. Krajcik and P. C. Blumenfeld, “Project-based learning,” in The Cambridge Handbook of the Learning Sciences, R. K. Sawyer, Ed. Cambridge, U.K.: Cambridge University Press, 2006, pp. 317-334.

[4] I. Sommerville, “Software Engineering,” Pearson, 10th ed., ISBN 978-0133943030, 2016.

[5] R. S. Pressman and B. R. Maxim, “Software Engineering: A Practitioner's Approach,” McGraw-Hill Education, 8th ed., ISBN 978-0078022128, 2015.

[6] K. Beck et al., Manifesto for Agile Software Development, agilemanifesto.org, Last accessed 28 October 2026.

[7] Next.js Documentation, Vercel, nextjs.org/docs, Last accessed 28 October 2026.

[8] React Documentation, Meta Open Source, react.dev, Last accessed 28 October 2026.

[9] Tailwind CSS Documentation, Tailwind Labs, tailwindcss.com/docs, Last accessed 28 October 2026.

[10] Supabase Documentation: Database, Auth and Storage, Supabase, supabase.com/docs, Last accessed 28 October 2026.

[11] PostgreSQL Documentation: Row Security Policies, The PostgreSQL Global Development Group, postgresql.org/docs, Last accessed 28 October 2026.

[12] PDF.js, Mozilla, mozilla.github.io/pdf.js, Last accessed 28 October 2026.

[13] OWASP Top 10: A01 Broken Access Control, OWASP Foundation, owasp.org, Last accessed 28 October 2026.

[14] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” National Institute of Standards and Technology, NIST SP 800-38D, 2007.

[15] Share on LinkedIn and Posts API, Microsoft Learn, learn.microsoft.com/linkedin, Last accessed 28 October 2026.

# ANNEXURES

## Annexure A: Individual Contributions

Table A.1: Individual contributions of the team members

| Member | Registration No. | Contribution |
|---|---|---|
| Mihir Sanghvi | 2427010544 | Technical lead. Set up the Next.js project and Vercel deployment; designed the Supabase schema, Row Level Security and database functions; built sign-in and sign-up, the student, supervisor and administrator portals, mentor registration, team requests, grading, the LinkedIn integration, the formatted announcement editor, the 100-point assessment with Final Grading, and the two-supervisor change of supervisor. |
| Krishna Agarwal | 2427010030 | Testing and front-end pages. Built the project, profile and faculty directory pages and the request review screens; wrote and ran test cases for every flow with the test accounts; checked layouts on different screen sizes and re-tested fixes. |
| Mahika Tuteja | 2427010439 | Testing, content and documentation. Prepared seed and test data and the content for deadlines and announcements; checked the scraped faculty data; drafted achievement categories and LinkedIn post templates; tested pasted announcements; collected screenshots and drafted parts of this report. |

## Annexure B: Deployment and Test Accounts

The portal is deployed at **pbl-sem-5-sigma.vercel.app**. Test accounts for three students, a team supervisor and an administrator were used during development and for the screenshots in this report. They were removed before submission, so only real university accounts remain on the live portal.